Universidad Militar Nueva Granada
Ingeniería en Multimedia · Inteligencia Artificial · 2026-2

App XOR: perceptrón multicapa con backpropagation

Parte C: agente, tipo de entorno, conclusiones y referencias

Autores

Integrante	Nombre	Código
A		
B		
C	David Felipe Alvarado Romero	

4. Agente

a. Data set: conjunto de datos a probar

Para entrenar la red se usa la tabla de verdad completa de la función XOR. Son cuatro patrones, cada uno con dos entradas binarias, x1 y x2, y una salida deseada *d*. La salida vale 1 solo cuando las dos entradas son distintas, y vale 0 cuando son iguales. No hay más casos posibles. Por eso los mismos cuatro patrones se usan para entrenar la red y también para comprobar lo que aprendió, ya que no existe ningún patrón nuevo que se pueda dejar aparte solo para la prueba.

Tabla 1. Conjunto de datos del XOR

Patrón	x1	x2	Salida deseada (d)
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

Si se ponen los cuatro puntos en el plano, los que tienen salida 1, (0,1) y (1,0), quedan en esquinas opuestas del cuadrado, y los que tienen salida 0 quedan en las otras dos esquinas, así que no hay una sola recta que los pueda separar (Universidad Militar Nueva Granada, 2026). Ahí está lo difícil del XOR.

b. Orden de ingreso de los datos al proceso de entrenamiento

Los patrones entran a la red en orden secuencial, el mismo de la tabla de verdad: (0,0) → (0,1) → (1,0) → (1,1). El entrenamiento es en línea, es decir, los pesos se corrigen patrón por patrón (Haykin, 2009). Después de cada patrón se hace el forward pass, se calcula el error, se retropropaga y se corrigen los pesos y los sesgos, y solo ahí entra el siguiente patrón.

En el resto del documento se habla de iteraciones y de épocas. Una **iteración** es la presentación de un solo patrón a la red, con su forward pass, su backpropagation y su actualización de pesos. Una **época** es la presentación de los cuatro patrones, así que cada época tiene cuatro iteraciones.

Se escogió el orden secuencial porque, junto con la semilla fija de los pesos iniciales, hace que el entrenamiento se pueda repetir y que siempre dé lo mismo, los mismos pesos finales, las mismas épocas y las mismas iteraciones, y así cualquier persona puede abrir la app y comprobar los valores que aparecen en el informe. La app también tiene un orden aleatorio por época, en el que los cuatro patrones se barajan al inicio de cada época. Con la semilla 1 ese orden no llegó a converger.

c. Error estimado para establecer el momento de parada del entrenamiento (medidas de desempeño)

Al terminar cada época se mide el error cuadrático medio (MSE) de la red sobre los cuatro patrones, con los pesos que tiene en ese momento (Goodfellow et al., 2016):

$$\text{MSE} = (1/4) \cdot \Sigma\, (d_i - y_i)^2, \text{ con i de 1 a 4}$$

En la fórmula, d_i es la salida deseada del patrón i y y_i es la salida que da la red para ese patrón. El entrenamiento se detiene con el primero de estos dos criterios que se cumpla:

1. **MSE ≤ 0.001.** La raíz de 0.001 es más o menos 0.03, lo que quiere decir que cada salida queda en promedio a unas tres centésimas de su valor deseado, muy lejos del umbral de 0.5 con el que se decide si la red responde 0 o 1.
2. **50 000 épocas.** Es un límite de seguridad. Sirve para el caso en que la red se quede atrapada en un mínimo local y el MSE ya no baje más.

Para saber qué tan bien quedó la red se miran cuatro medidas de desempeño:

Tabla 2. Medidas de desempeño

Medida	**Qué muestra**	**Valor buscado**
MSE final	Qué tan cerca quedan las salidas de los valores deseados	≤ 0.001
Patrones correctos	Cuántas salidas quedan del lado correcto del umbral ($y \geq 0.5$ cuenta como 1)	4 de 4
Épocas	Cuántas veces se recorrió el data set hasta parar	Las menos posibles
Iteraciones	Cuántas veces se corrigieron los pesos (épocas × 4)	Las menos posibles

Con la configuración del proyecto la red paró por el primer criterio. Llegó a un MSE de 0.00099964 en la época 3918, con 15 672 iteraciones y los 4 patrones bien clasificados.

d. Tipo de agente

Mientras se entrena, la red funciona como un **agente que aprende** dentro de un esquema de aprendizaje supervisado (Russell y Norvig, 2021). Un agente que aprende tiene cuatro partes, y cada una se puede ver en la red del XOR:

Tabla 3. Partes del agente que aprende en la red del XOR

Parte del agente	**Qué es en la red del XOR**
Elemento de desempeño	La red 2-2-1 con sus pesos actuales, que da la salida y con el forward pass
Crítico	La comparación entre la salida y y la salida deseada d, con el error $(d - y)$ y el MSE
Elemento de aprendizaje	La retropropagación del error, que corrige pesos y sesgos con la regla delta generalizada
Generador de problemas	El data set y el orden de ingreso, que deciden qué patrón entra en cada iteración

Ya entrenada, la red se comporta como un **agente reactivo simple**. Percibe las dos entradas y responde con una función fija, que son los pesos finales, sin memoria de lo que entró antes y sin planear nada. Lo que aprendió queda guardado en los pesos.

5. Tipo de entorno

a. Explicación

Para la red, el entorno es el problema XOR: los cuatro patrones de entrada con sus salidas deseadas. La red ve las dos entradas completas de cada patrón, así que el entorno es totalmente observable. Es determinista porque la misma entrada siempre tiene la misma salida correcta, y es estático porque el data set no cambia ni mientras la red entrena ni mientras responde. También es episódico. Cada patrón se clasifica por separado y la respuesta a uno no depende de lo que se respondió en el anterior. Es discreto, porque solo hay cuatro entradas posibles con valores de 0 o 1 y dos clases de salida, y además hay un solo agente, que es la red, y el entorno es conocido, ya que las salidas correctas se saben desde antes y son las que guían el aprendizaje supervisado.

Es un entorno sencillo a propósito. Justo por eso deja ver con claridad lo que hace la capa oculta.

b. Tabla relacionada con la explicación

Tabla 4. Tipo de entorno del XOR

Propiedad	**Clasificación**	**Por qué**
Observabilidad	Totalmente observable	La red recibe x1 y x2 completos en cada patrón
Número de agentes	Un solo agente	Solo la red actúa sobre el problema
Determinismo	Determinista	Cada entrada tiene una sola salida correcta, la de la tabla de verdad
Episodios	Episódico	Clasificar un patrón no depende de lo que se respondió en el anterior
Dinámica	Estático	Los 4 patrones no cambian ni en el entrenamiento ni en la prueba
Estados y acciones	Discreto	4 entradas posibles con valores 0 y 1, y 2 clases de salida
Conocimiento	Conocido	Las salidas deseadas se saben desde antes y guían el aprendizaje

8. Conclusiones

1. **El perceptrón multicapa sí resuelve el XOR.** Un perceptrón de una sola capa no puede hacerlo, porque el XOR no es linealmente separable (Minsky y Papert, 1969). Con una capa oculta de dos neuronas sigmoides y una tasa de aprendizaje de 0.5, la red pasó de acertar 2 de

los 4 patrones con los pesos iniciales, con un MSE de 0.2533, a acertar los 4 con un MSE final de 0.00099964, después de 3918 épocas y 15 672 iteraciones.

2. **La capa oculta cambia la forma en que la red ve las entradas.** Con los pesos finales, h1 solo se activa con la entrada (1,0) y h2 solo se apaga con la entrada (0,1). Cada neurona oculta traza una recta en el plano (x1, x2), y la neurona de salida junta esas dos rectas para separar las dos clases, cosa que una sola recta no puede hacer y que es justo la idea que mostraron Rumelhart et al. (1986) con la retropropagación. La frontera de decisión deja de ser lineal.

3. **El error no baja de forma pareja.** En las primeras 500 épocas, más o menos, el MSE se queda cerca de 0.25, que es lo que da la red cuando responde alrededor de 0.5 en todos los patrones. Después cae rápido. La red acierta los 4 patrones desde la época 978, el MSE baja de 0.1 en la época 1003 y de 0.01 en la 1393, y el resto de las épocas, hasta la 3918, sirve para acercar las salidas a 0 y a 1 hasta llegar al objetivo de 0.001.

4. **El resultado depende de los pesos iniciales, de la tasa de aprendizaje y del orden de ingreso.** Se probaron las semillas de la 1 a la 20 con $\eta = 0.5$. De esas, 14 convergieron, entre 3480 y 6798 épocas, y 6 se quedaron atrapadas en un mínimo local, con un MSE entre 0.125 y 0.167 y solo 3 de 4 patrones correctos. Al cambiar la tasa de aprendizaje con la semilla 1, $\eta = 1.0$ convergió en 1947 épocas y $\eta = 2.0$ en 1029, pero $\eta = 0.1$ y $\eta = 0.25$ se quedaron en un mínimo local. Con orden aleatorio y la misma semilla, la red tampoco convergió. Por eso el límite de 50 000 épocas hace falta como segundo criterio de parada, y queda la pregunta de si con más neuronas en la capa oculta la red dejaría de caer en esos mínimos locales, algo que en este trabajo no se probó.

5. **Dividir el algoritmo en partes dejó trabajar a los tres en paralelo.** El forward pass y el MSE (parte A), la retropropagación y la actualización de pesos (parte B), y el data set, el ciclo de entrenamiento y la interfaz (parte C) se conectan con funciones que tienen parámetros acordados entre los tres. Todo el proceso inteligente se programó a mano. NumPy solo se usó para operar matrices (Harris et al., 2020) y matplotlib para hacer las gráficas (Hunter, 2007), sin ninguna librería de redes neuronales.

9. Referencias

Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep learning*. MIT Press.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Haykin, S. (2009). *Neural networks and learning machines* (3.ª ed.). Pearson.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Minsky, M. y Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.

Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536. https://doi.org/10.1038/323533a0

Russell, S. y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Universidad Militar Nueva Granada. (2026). *Aplicaciones redes neuronales 2026-2* [Guía de la asignatura Inteligencia Artificial].